



UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

nJavaCalendar

Name: Incze
Antonia
Group: 30435/2

Table of Contents

Deliverable 1.....	3
Project Specification	3
Functional Requirements	3
Use Case Model	3
Use Cases Identification:	3
UML Use Case Diagrams.....	5
Supplementary Specification	5
Non-functional Requirements	5
Design Constraints	5
Glossary.....	6
Deliverable 2.....	6
Domain Model	6
Architectural Design	6
Conceptual Architecture	6
Package Design	7
Component and Deployment Diagram	8
Deliverable 3.....	8
Design Model.....	8
Dynamic Behavior	8
Class Diagram	9
Data Model.....	10
System Testing.....	11
Future Improvements.....	12
Conclusion.....	12
Bibliography	13

Deliverable 1

Project Specification

nJavaCalendar is a Time Management Application designed to help users stay on top of their schedule with an interactive and easy to use UI.

Functional Requirements

The system shall:

- Allow users to view a weekly schedule displaying time blocks aligned by time
- Allow users to add recurring schedule blocks, representing for example weekly lectures/work
- Allow users to add one-time events for specific dates
- Display time blocks in an alignment by time, where position and size reflect start and end time
- Allow users to edit or delete existing time blocks
- Allow users to assign colors to time blocks
- Allow users to view details of a time block
- Allow users to create and view weekly tasks with no determined time, in the form of a checked list
- Allow users to drag and drop tasks in this list

Use Case Model

Use Cases Identification:

Use-Case: View weekly schedule

Level: User goal

Primary Actor: User

Main success scenario:

- User opens application
- System requests weekly schedule from backend
- System displays 7-day calendar
- Time blocks are shown aligned by time
- User can visually inspect their schedule

Extensions:

- If no events exist, the system displays an empty schedule

Use-Case: Add time block

Level: User goal

Primary Actor: User

Main success scenario:

- User opens "Add Event" form
- User inputs title, day/date, start time, end time and color
- System validates input
- System stores the new block
- System updates calendar view

Extensions:

- Invalid time format results in a validation error

Use-Case: Edit/Delete time block

Level: User goal

Primary Actor: User

Main success scenario:

- User clicks on an existing block
- Edit flow: user modifies details, system validates input and updates block
- Delete flow: user confirms deletion. System removes block
- Calendar refreshes

Extensions:

- Invalid edit results in error
- If a deletion is cancelled, no action is taken

Use-Case: Add Task

Level: User goal

Primary Actor: User

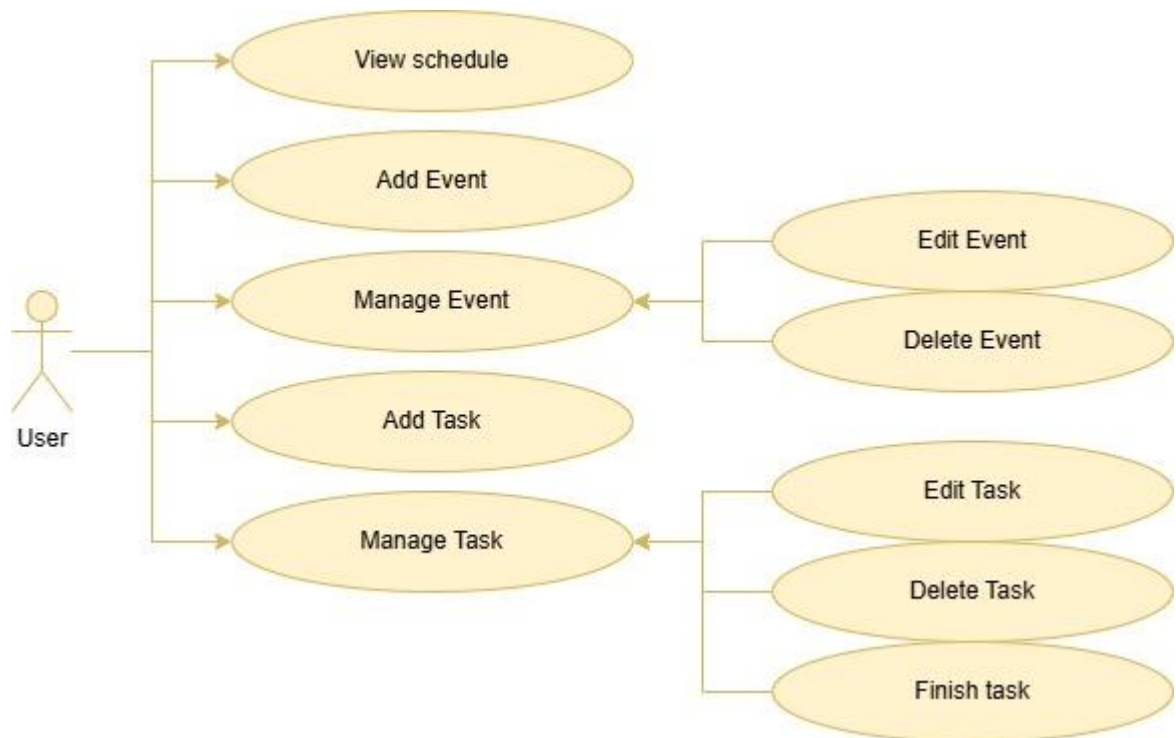
Main success scenario:

- User opens "Add Task" form
- User inputs task description
- System stores task
- Task list refreshes

Extensions:

- Task order may be changed through drag and drop
- Checked tasks disappear at the end of the week
- Possible extension: task history

UML Use Case Diagram



Supplementary Specification

Non-functional Requirements

- The system shall provide an intuitive and visually clear interface for managing schedules
- The system shall load and display the weekly schedule within 1 second under normal conditions
- The system shall prevent inconsistencies such as overlapping time blocks
- The system shall follow a layered architecture

Design Constraints

1. Programming languages
 - Backend: Java Spring Boot
 - Frontend: React
2. Data Storage
 - Currently in-memory, to be extended to database ASAP
3. Development Tools
 - IntelliJ IDEA
 - Node.js & npm

4. Frontend constraints:

- No external UI frameworks, susceptible to change

Glossary

Time Blocks – A scheduled activity with start and end time

Recurring Block – A time block repeated weekly (e.g. lecture)

One-Time Event – A single occurrence event (e.g meeting)

Schedule – A collection of time block across 7 days

Day Schedule – All time blocks for a specific day

DTO – Object used to transfer data between front and backend

View – UI representation of schedule

Edit Mode – UI state allowing modification of blocks

Deliverable 2

Domain Model

The application centers on the idea of a user managing a weekly schedule composed of time blocks and task items. Based on the project specification, the most important domain concepts are recurring schedule blocks, one-time events, and tasks.

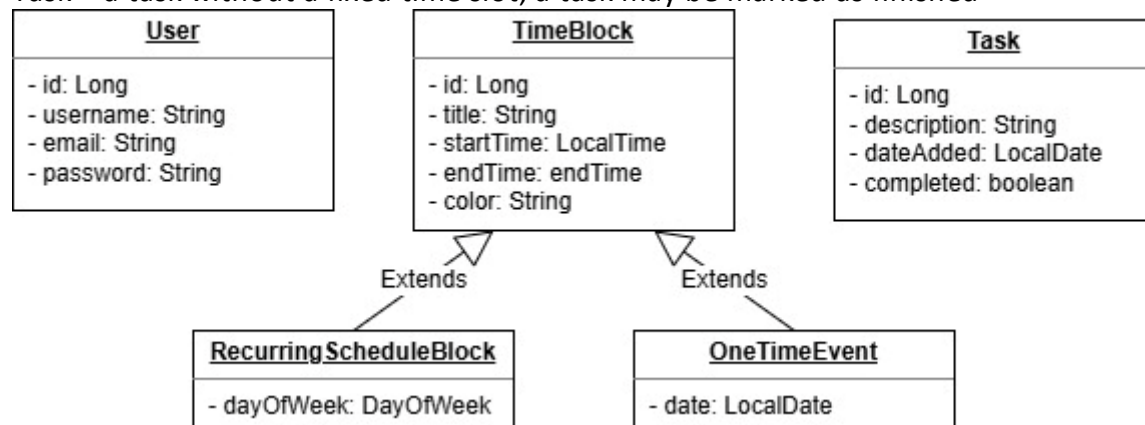
User – person using the application; a user owns one schedule and one task collection

Time Block – scheduled activity with a start and end time; it also contains attributes such as title and color

Recurring Time Block – a special time block that represents recurring activities such as school hours or work shifts

One Time Event – a time block that represents a non-recurring event

Task – a task without a fixed time slot; a task may be marked as finished

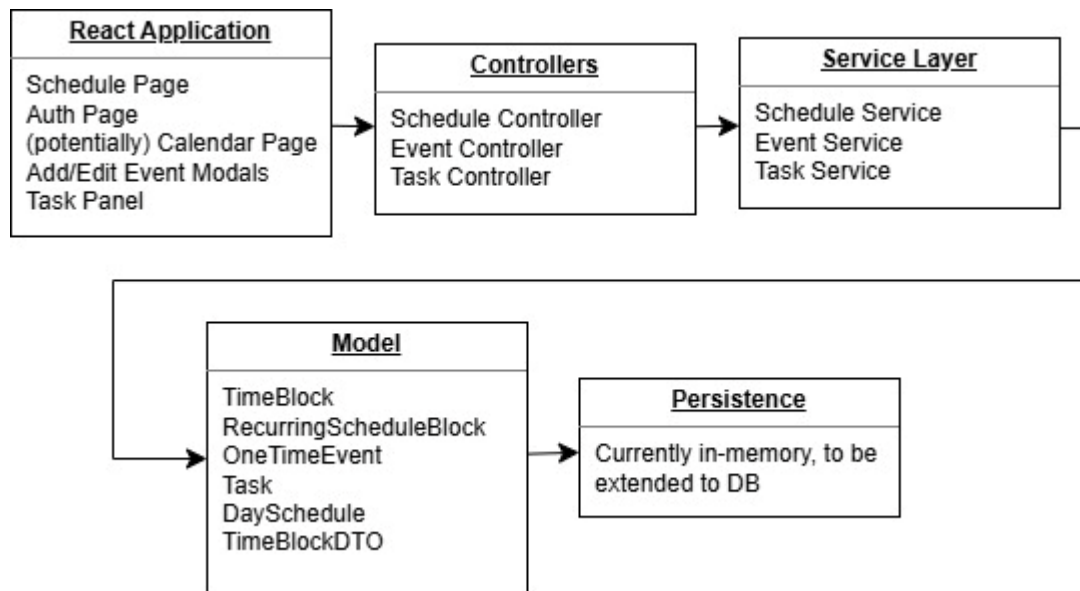


Architectural Design

Conceptual Architecture

Frontend: React application

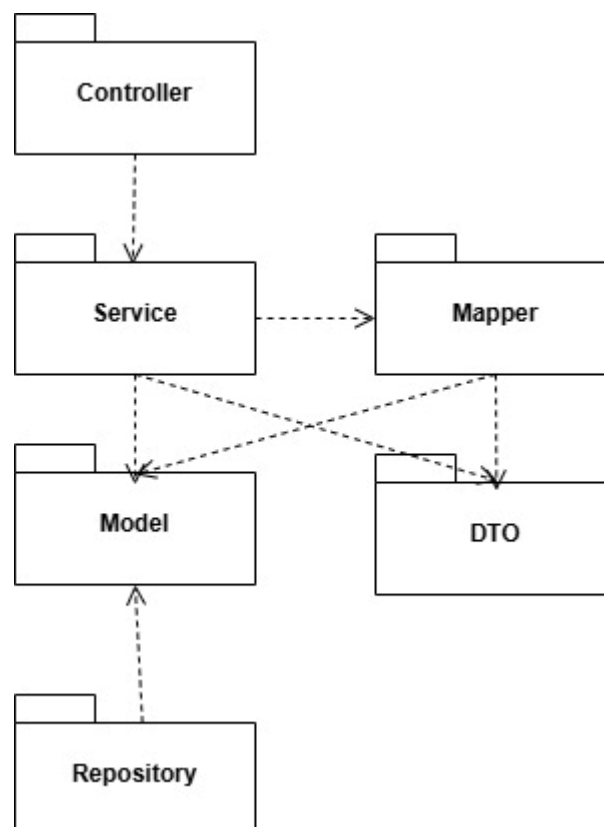
Backend: Spring Boot REST API, Layered Architecture



This is consistent with the project's non-functional requirement to follow a layered architecture.

Package Design

The package design should reflect the layered architecture.



Component and Deployment Diagram

At the current stage, the application can be deployed locally with:

- a browser running the React frontend,
- a Spring Boot server running the backend,

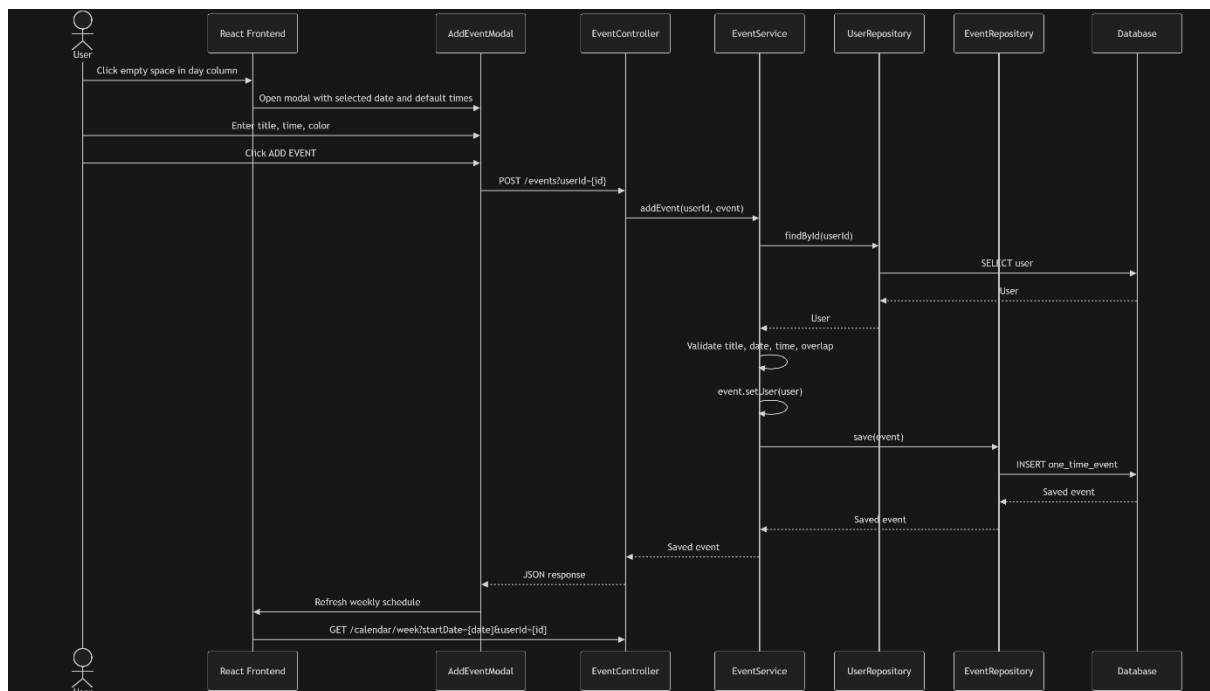
Deliverable 3

Design Model

Dynamic Behavior

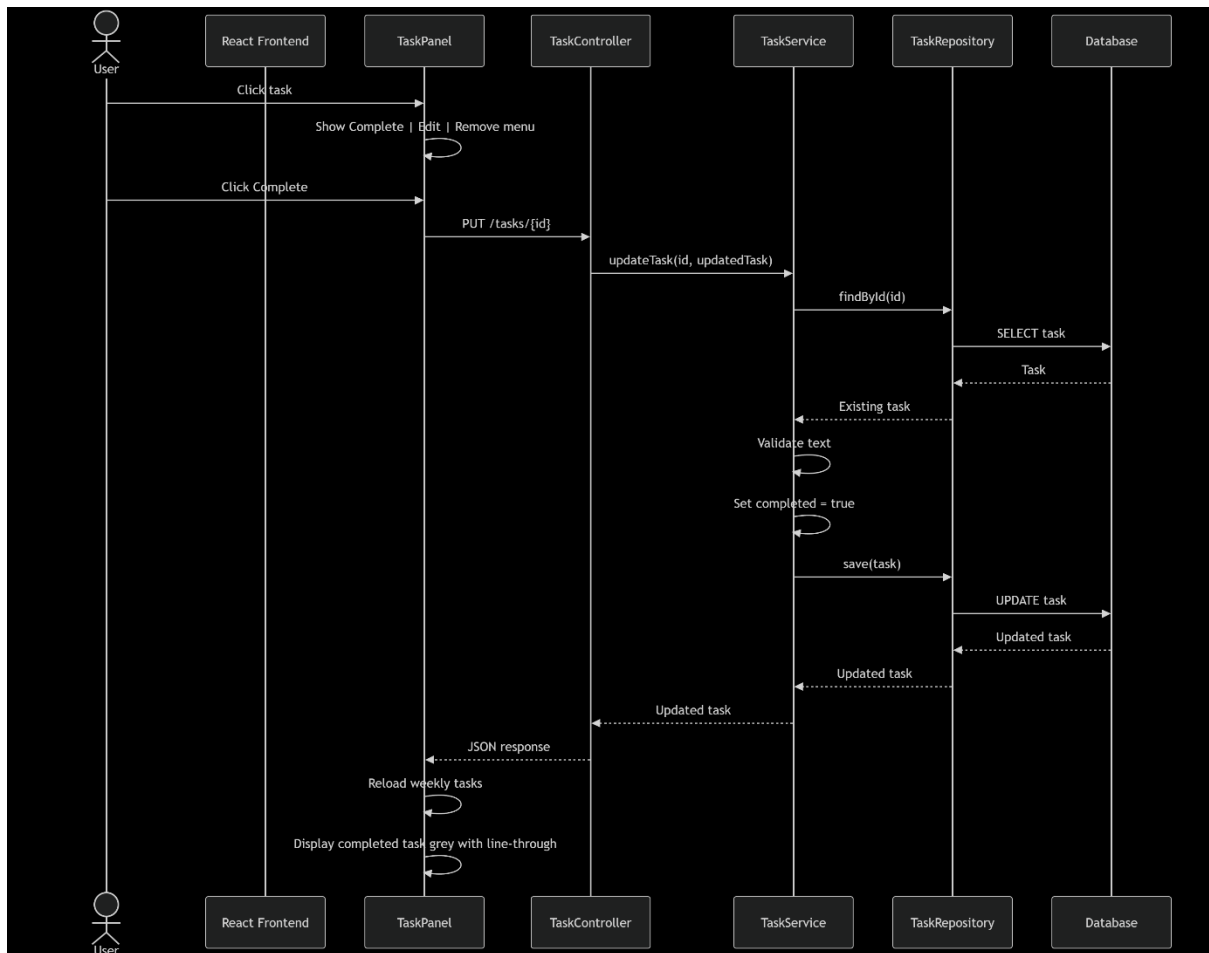
Sequence Diagram 1: Add One-Time Event

The user clicks an empty space in a day column, fills the Add Event modal, leaves “Recurring” unchecked, and submits the form.

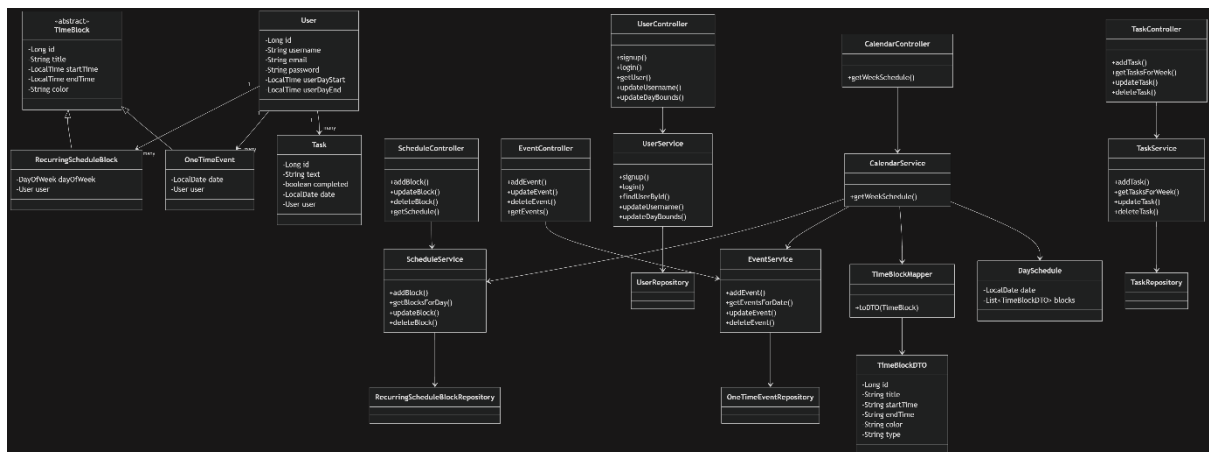


Sequence Diagram 2: Complete Task

The user clicks a task, opens the task menu, and selects “Complete”.



Class Diagram



GoF Pattern Applied

1. Façade

The system applies the Facade pattern through CalendarService.

CalendarService provides a simplified interface for generating the weekly schedule. Instead of the controller calling multiple services directly, it delegates the process to CalendarService.

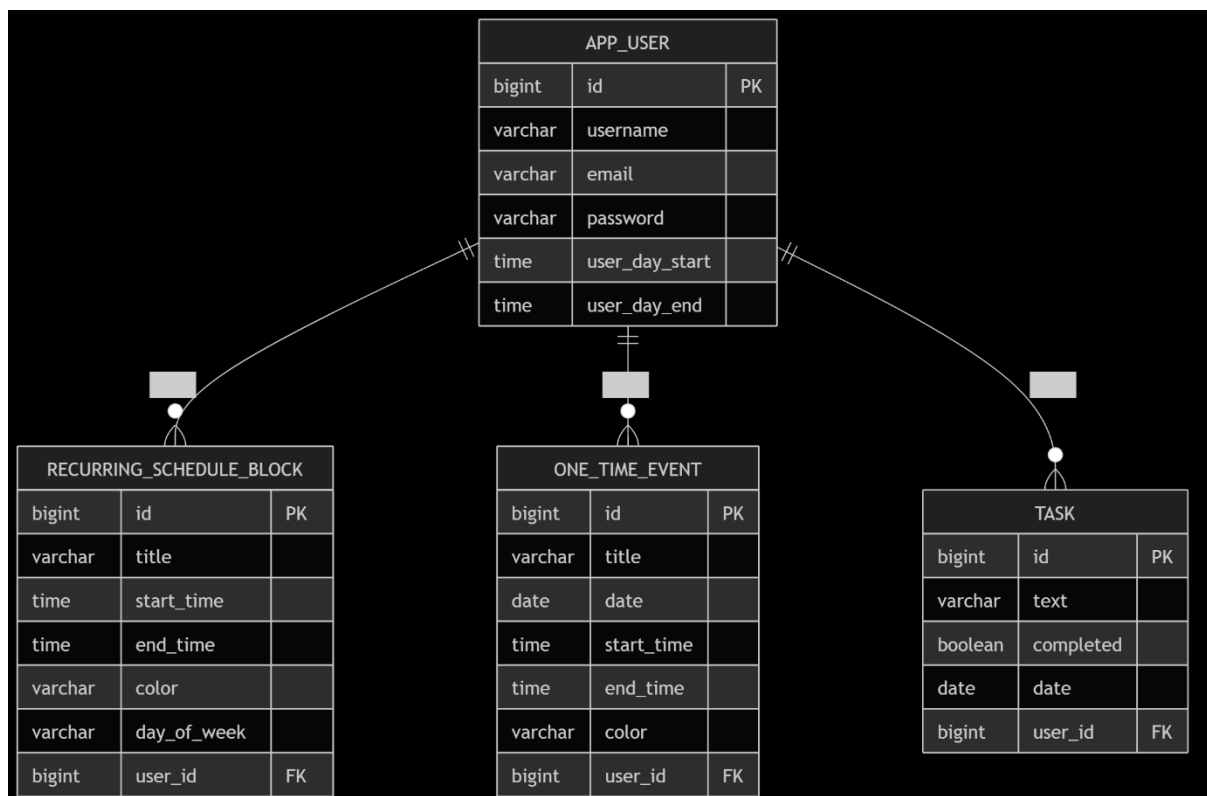
This choice was made to hide the complexity of multiple sources from the controller, giving it just one, clean operation: `getWeekSchedule(userId, startDate)`

2. Data Transfer Object (DTO)

The system also applies the DTO pattern through TimeBlockDTO, DaySchedule, UserResponse, SignupRequest, LoginRequest. This is to provide the frontend with an appropriate shape, rather than a raw entity.

For example, TimeBlockDTO allows both RecurringScheduleBlock and OneTimeEvent to be displayed uniformly in the calendar.

Data Model



Data Rules

1. A user can own many recurring schedule blocks.
2. A user can own many one-time events.
3. A user can own many tasks.
4. A recurring block must have a valid dayOfWeek.
5. A one-time event must have a valid date.

6. A task must have non-empty text.
7. $startTime < endTime$.
8. Events and recurring blocks owned by the same user cannot overlap within the same day.
9. Completed tasks disappear from the current task view after their week passes, but remain stored in the database.
10. Incomplete tasks persist into future weeks.

System Testing

1. Unit testing

Unit testing refers to validating isolated service methods without running the full application. This is to ensure that logic works independently from the frontend and the database.

Example test cases for adding recurring schedule blocks: **ScheduleService.addBlock()**

- >Empty title => IllegalArgumentException("Empty title is not accepted")
- >Empty start/end time => IllegalArgumentException("Start and end time are required")
- >End time earlier than start time => IllegalArgumentException("Invalid start and end times")

2. Integration testing

Verify that layers communicate correctly.

Example: **POST/events?userId=1**

- >Verify that the event is saved with the correct user
- >Send GET/calendar/week
- >Verify that the event appears in the weekly response

3. UI Testing

Manual frontend testing checks behavior on the interface. This is to verify that the user interface acts as expected.

Examples:

Click a day column

Expected behavior > modal for adding an event for that day appears

Add an event

Expected behavior > Event data is validated and given valid data, the modal closes and the new event shows up in the weekly view at its respective time slot

Change schedule hours

Expected behavior > Schedule hours are validated and the schedule refreshes to reflect the new day start/end hours

Future Improvements

1. Secure Authentication (most important)

A real login system would need to be improved by adding proper password hashing and token-based authentication for security reasons

2. Improved Time Zone Support

The current application uses the browser's local date and time to account for users in different time zones, but a feature could be to store a user's preferred time zone for users who travel often, work remotely or have events across different time zones

3. Vacation / Break Periods

The original project idea included vacation time. This could be added so recurring schedule blocks disappear during selected date ranges

4. Improved Overlap Handling

Currently, overlapping blocks are completely rejected, however, realistically, a user could have overlapping events they may need to keep track of, so a future version of the application could support overlaps by displaying overlapping blocks side by side

5. Monthly Calendar View

Originally, the application was planned to support weekly and monthly scheduling. The monthly view was scrapped due to time constraints, however a future version could be expanded with a full month view, allowing a user to create notes per day

6. Color Palette Customization

Users could create a personal color palette for events and recurring blocks that then show up in a quick color selector rather than the full color scope. This would help for color consistency

Conclusion

The nCalendar application provides a structured solution for personal time management by combining weekly scheduling and tasks. The system allows each user to create an individual schedule, define their preferred visible day hours, and manage weekly tasks and events by interacting with a React frontend, connected to a Spring Boot REST backend.

From a design perspective, the project makes use of a layered architecture that separates the frontend, controller layer, service layer, repository layer and database model. This separation aims to improve maintenance and allows each part of the system to evolve independently. The use of DTOs provides the frontend with responses while protecting the internal structure of the entities.

Overall, the project serves as a foundation for a full-stack scheduling application and demonstrates software design principles such as modularity, encapsulation, data persistence and separation of concerns.

Bibliography

1. Spring Initializr
<https://start.spring.io/>
2. Spring Boot Reference Documentation
<https://docs.spring.io/spring-boot/>
3. Spring Data JPA Reference Documentation
<https://docs.spring.io/spring-data/jpa/reference/>
4. H2 Database Documentation
<https://www.h2database.com/html/main.html>
5. React Documentation
<https://react.dev/>
6. Oracle Java Documentation
<https://docs.oracle.com/en/java/javase/>